



PROJETO 3 – MINIX

OBJETIVOS

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
 - Compilar e modificar o kernel
- Criação de processos, IPC, gerenciamento de sinais, semáforos
 - Processos órfãos e zumbis
 - Criação de threads
- Gerenciamento de I/O no MINIX
 - Criar uma chamada de sistema;
 - Criar uma chamada de *kernel* e usá-la via uma chamada de sistemas (item anterior);
 - Criar um *character device*.

INFORMAÇÃO ADMINISTRATIVA

Este projeto pode ser realizado em grupos de no máximo 3 alunos. Observe que a execução bem sucedida deste projeto é importante, considerando que este irá oferecer as bases para projetos futuros, quanto a interação com o SO MINIX. Um relatório descritivo dos procedimentos realizados e as respostas às questões presentes no final deste projeto devem ser entregues usando o modelo de relatório técnico-científico disponível no site da disciplina. **O relatório deve ser entregue impresso em duas vias para o professor da disciplina** até o dia 18/10/2018, no horário da aula. Atenção para o prazo de tarefas individuais especificadas no decorrer desta prática. Com exceção da tarefa extraclasse, as demais possuem uma sugestão de data de execução para os alunos de modo a se manter um cronograma de estudo apropriado para a data da avaliação 3.

OBS.:

- Como material adicional, leia o artigo “*How to Add a New System Call for Minix 3*” disponível no site do curso ([How_to_add_system_call.pdf](#)).
- As modificações nos fontes do kernel **devem ser executadas usando a conta de root**.

TAREFA EXTRACLASSE: FAMILIARIZAÇÃO COM A ESTRUTURA DO SO (Entrega em: 04/10/2018)

Para a execução desta tarefa, o aluno deverá utilizar o seguinte material:

- **Módulo 1:** introdução ao MINIX (11 slides);

- **Módulo 2:** *overview* de processos no MINIX (172 slides).

O objetivo desta tarefa é instigar o aluno ao aprendizado sobre o SO MINIX, com relação a suas características, de modo mais aprofundado do que apenas o aspecto prático envolvido na atividade prática a seguir. A lista de exercícios (TAREFA EXTRACLASSE - Lista de exercícios.pdf) decorrente desta atividade terá seu gabarito publicado após a entrega da lista de exercício manuscrita no dia 04/10/2018. Vale observar que algumas respostas de exercícios estão no material teórico disponível no texto desta prática de laboratório. Alunos que não entreguem esta atividade no prazo perderão a nota correspondente na avaliação do trabalho final.

TAREFA 1: INSTALAÇÃO DO SISTEMA (25/09/2018)

É recomendável que você use algum software para máquina virtual. Exemplos incluem o Bocks, Qemu, VirtualBox, VMware, etc. Para este projeto recomenda-se o uso do **VirtualBox**, por ser *open source* e já ter alcançado um estágio de desenvolvimento estável.

Você é encorajado a instalar o MINIX do “zero”. Adquira a imagem no site do MINIX e instale na máquina virtual. A prática foi planejada originalmente para a versão **R3.1.8-r8398** do MINIX, a qual está disponível com o docente (a última versão é a 3.3.0, disponível no site do MINIX). É recomendável que o aluno tente realizar a prática e adequá-la a versão mais recente do SO MINIX, entretanto, não haverá qualquer prejuízo caso o aluno escolha fazer a atividade com a versão mais antiga do SO. Observe a documentação disponível na página do curso e no site do MINIX.

OBS.: para o curso, recomenda-se uma instalação com 2 GB de espaço livre em disco e pelo menos 128 MB de memória RAM (execução em linha de comando).

Para esta atividade, documente as seguintes atividades:

- Instalação do SO MINIX;
- Configuração da rede da máquina virtual (demonstre que a rede funciona);
- Instalação do ssh (cliente e servidor);
- Acesso a máquina virtual via ssh (acesso remoto). Isso pode ser feito a partir do sistema operacional hospedeiro (i.e. sistema no qual o VirtualBox está instalado);
- Criação de um arquivo *teste.txt* (computador hospedeiro) e transferência para a máquina virtual (usando o servidor de ssh recém instalado).
- Criação de uma conta de usuário para você;
- Instalação do *bash*;
- Configuração do *bash* como o shell padrão para o seu usuário (ao final, use o arquivo *.profile* disponível no site do curso).

TAREFA 2: RECOMPILANDO O KERNEL DO MINIX (27/09/2018)

Nesta atividade você irá recompilar o *kernel* do MINIX. Para tal, vá para o diretório `/usr/src/tools` e chame o comando **make**. Verifique as opções que aparecem no arquivo `Makefile` e descubra como um kernel é compilado e instalado.

Nota: uma vez instalado o kernel e reiniciado o MINIX, é importante selecionar a segunda opção “*Start custom kernel*” na tela de início. De outra forma, o MINIX não irá executar o kernel mais recentemente compilado.

Estando apto a construir um novo kernel, finalmente é hora de fazer uma pequena modificação no SO. Primeiro, entretanto, você deve fazer uma cópia “limpa” da árvore de diretórios do kernel. Isto é necessário por dois motivos. Primeiro, quando posteriormente você fizer mudanças que não funcionem e quando precisar de uma referência para consultar, a árvore com os fontes originais estarão lá (percebe agora, por que fazer uma instalação de 2 GB?). Segundo, e mais importante, tal referência é necessária para que você possa criar *patches* (veja abaixo). Assim, faça “`cp -r /usr/src/ /usr/src.clean`”, e você pode continuar.

Dica: para evitar erros ou impossibilitar o uso de seu MINIX, você também pode usar o recurso de *snapshot* do VirtualBox.

Para esta prática, implementaremos uma pequena modificação no *kernel*. Você deve **imprimir o nome de cada arquivo que está sendo executado pelo SO**. Para tanto, você deve localizar os fontes do *kernel* que implementam a chamada de sistema **exec**. Então, no ponto correto, você deve inserir uma chamada `printf` que coloque a string “executando” seguida pelo nome do arquivo que está sendo executado. Por exemplo: quando você digita “ls” no shell, você deve ver a string “MSG: executando ls” impressa no console. Isto deve em verdade, requerer apenas uma única linha a ser adicionada, mas isto deve mostrar algumas características de depuração (*debug*) muito simples a você.

Uma vez que você faça essa modificação, recompile o *kernel*, instale-o e reinicie o sistema. Caso você seja bem sucedido, você deverá poder ver uma série de mensagens que permitam que você veja quais programas o sistema operacional está executando (*launching*).

Dica: leia o artigo “How to add a system call and System call sequence for MINIX3” (disponível na área de material didático) para entender melhor como as chamadas de sistema são servidas no SO MINIX. Uma vez que você entenda o mecanismo, você pode facilmente localizar o lugar para

inserção do código.

Sua última tarefa é submeter sua mudança ao professor. Para lidar com grandes sistemas de software (e.g. sistemas operacionais como o MINIX ou o Linux), não faz sentido enviar o código fonte completo para cada pequena mudança. Para isto, foram inventados os *patches*. Um *patch* captura as mudanças entre dois arquivos diferentes (arquivo antigo e arquivo novo). Dado o arquivo antigo e o *patch*, é possível usar o programa `patch` para criar o arquivo novo. É exatamente isso que você deve fazer. Dessa forma, ao terminar a sua modificação do kernel (incluindo recompilar, instalar e testar tudo), você deve ir ao diretório `/usr` e produzir um *patch* para cada arquivo que você modificou. Para isto, você deve chamar o comando `diff -c (path/to/oldfile) (path/to/newfile)` e armazenar a saída em um arquivo chamado `seunome.patch`. Por exemplo, ao fazer um *patch* do arquivo `/usr/src/servers/inet/clock.c`, você deve primeiro mudar para o diretório `/usr` e então executar o comando `diff -c src/clean/servers/inet/clock.c src/servers/inet/clock.c`. Você pode testar se o seu *patch* funcionou pela criação de outra cópia da árvore de diretórios (e.g. `/usr/test`), ir ao diretório `/usr/test` e executar o comando `patch -p1 < seunome.patch`. Isto deve modificar o arquivo `clock.c` no diretório `/usr/test` da maneira esperada. Além disso, é assim que os *patches* dos alunos serão testados. Espera-se que eles possam ser aplicados de maneira correta (sem quaisquer erros ou *warnings*).

Dica: use redirecionamento de saída. Dessa forma é possível escrever a saída do comando `diff` diretamente em um arquivo texto.

TAREFA 3 (gerenciamento de processos): criação de processos, IPC, gerenciamento de sinais, semáforos (02/10/2018 e 03/10/2018)

No Projeto 1, trabalhamos especificamente com *threads*. Durante esse projeto, no contexto do MINIX, trabalharemos com chamadas de sistemas para a criação e comunicação de processos.

- (a) (02/10/2018) Como funciona a chamada de sistema `fork`. Explique no contexto do programa `fork-ex.c`. Como você pode ter a certeza que o programa `fork-ex.c` está de fato funcionando? Demonstre.
- (b) (02/10/2018) Modifique o programa `fork-ex.c` para que este realize a criação de 10 processos filhos. Modifique a função `void do_nothing (void)` de modo que esta imprima para cada execução de processo filho o `pid` do processo pai e o `pid` do processo filho (OBS. O PROTÓTIPO DESTA FUNÇÃO NÃO DEVE SER MODIFICADO). Além

disso, crie mecanismos em seu programa para permitir a coleta de dados dos 10 processos simultaneamente por meio do comando `ps`.

- (c) (02/10/2018) Escreva um programa o qual deve **criar 3 processos**: 2 escritores e 1 leitor. Os escritores devem escrever em um *pipe* mensagens de tamanho pré-definido (`sizeof(PACKET) bytes`). O leitor deve ler as mensagens inseridas pelos escritores, imprimir as mensagens e identificar o processo escritor que a originou. Para este exercício utilize a definição de um pacote conforme segue:

```
typedef struct {  
    int id; /* Deve guardar o pid do processo originário */  
    char data[256];  
    int lastone; /* Usado para identificar a última struct inserida no  
                pipe */  
} PACKET;
```

Use a implementação com FIFO, tendo em vista que o PIPE a ser criado não é utilizado exclusivamente para a comunicação entre processo pai e processo filho. Além disso, sua solução deve ser capaz de generalizar o caso de múltiplos escritores e um leitor. Referência: [Beej's Guide to Unix Interprocess Communication](#).

- (d) (02/10/2018) Como você implementaria o exercício da letra (c) usando *sockets* TCP? Forneça um exemplo de código usando um único leitor e um escritor. Use a definição de `PACKET` fornecida. Como generalizar a solução para múltiplos escritores? É possível? Explique. **Dica:** quem será o cliente? Quem será o servidor?
- (e) (04/10/2018) Considere as modificações implementadas na letra (b). Pesquise o funcionamento da macro `WEXITSTATUS(status)` em C e modifique seu programa de modo que, para cada um dos 10 processos filhos criados, eles possuam um status de saída diferente e controlado pelo usuário (ex. você pode especificar o status de saída a partir da linha de comando para cada um dos 10 processos ou simplesmente inserir no código um valor de saída diferenciado para cada processo filho).
- (f) (04/10/2018) Analise o código `ctrlc.c`. Explique o que este código realiza. Teste este código no MINIX e no Linux. Você observou alguma diferença quanto a execução? Explique. Referência: [Beej's Guide to Unix Interprocess Communication](#).

OBSERVAÇÃO:

Para compilar seus novos programas, vc pode inserir linhas apropriadas no arquivo Makefile. A forma mais simples de compilação seria:

```
$gcc exemplo.c -o exemplo
```

o parâmetro “`-o exemplo`” indica que o arquivo binário gerado (output) deverá se chamar

exemplo.

TAREFA 4 (gerenciamento de processos): Processos órfãos (04/10/2018)

TEÓRICO: Um processo órfão é um processo cujo processo pai terminou ou foi terminado.

Processos órfãos desperdiçam recursos e podem potencialmente complicar a operação de um servidor. Entretanto, existem algumas soluções para o problema de processos órfãos:

1. Exterminar o processo é a técnica mais comumente usada. Neste caso o processo órfão é terminado.
2. Expiração é uma técnica na qual cada processo é alocado por um certo período de tempo para que possa terminar antes que o processo seja finalizado. Caso necessário, um processo pode pedir mais tempo para terminar antes que seu tempo expire.

Ao se tornar órfão em um sistema *Unix-like*, um processo é automaticamente adotado pelo processo “init”. Esta operação é chamada *re-parenting* (em uma tradução livre, apadrinhamento) e ocorre por padrão. Embora o processo tenha o processo `init` como seu pai, ele ainda é chamado processo órfão pois o processo que o gerou originalmente não existe mais.

Simule a questão de processos órfãos, usando o comando `kill` no processo pai da tarefa 1(b) desta lista. Demonstre o *re-parenting* dos processos filhos e descreva o método usado para realizar este experimento.

Dica: para a coleta de dados use o comando `ps tree`.

TAREFA 5 (gerenciamento de processos): Processos zumbis (zombie.c) (04/10/2018)

TEÓRICO: No Unix e SOs *Unix-like*, um processo zumbi ou defunto é um processo que completou sua execução mas ainda possui uma entrada na tabela de processos. Esta entrada ainda é necessária para permitir ao processo que iniciou o processo (agora zumbi) a leitura de seu status de saída. O termo processo zumbi deriva da definição comum de zumbi - uma pessoa não morta. Nos termos desta metáfora, o processo filho já morreu, mas ainda não foi enterrado.

Quando um processo termina, toda a memória e recursos associados a ele são desalocados de modo que possam ser usados por outros processos. Entretanto, a entrada do processo na tabela de processo ainda perdura. O pai pode ler o status de saída do filho pela execução da chamada de sistema `wait`, momento no qual o zumbi é removido. A chamada `wait` pode ser executada em código sequencial, mas é comumente executada em um *handler* pelo sinal `SIGCHLD`, o qual o pai recebe sempre que um filho morre.

Após a remoção do zumbi, seu ID de processo e entrada na tabela de processo pode então ser reutilizado. Entretanto, se um pai falha na chamada `wait`, o zumbi será deixado na tabela de processo. Em algumas situações, isto pode ser desejado, por exemplo, se o pai cria outro processo

filho, ele se assegura que não irá alocar o mesmo ID de processo.

Um processo zumbi não é o mesmo que um processo órfão. Um processo órfão é um processo que ainda está executando, mas cujos pais já morreram. Eles não se tornam processos filhos; ao invés disso, eles são adotados pelo `init` (ID de processo 1), o qual espera por seus filhos.

Zumbis podem ser identificados na saída do comando Unix `'ps'` pela presença de um “z” na coluna STAT. Zumbis que existem por mais do que um pequeno intervalo de tempo, tipicamente indicam um *bug* no programa do processo pai, ou apenas uma decisão incomum ao sepultar filhos (veja o exemplo). A presença de uns poucos zumbis não é algo preocupante em si, mas pode indicar um problema que se tornaria sério sob cargas excessivas. Como não existe memória alocada a processos zumbis, exceto para a tabela de processo, a preocupação primária com muitos zumbis não é ficar sem memória, mas ao invés disso, ficar sem números para o ID do processo.

Para remover zumbis de um sistema, o sinal `SIGCHLD` pode ser enviado ao pai manualmente, usando o comando `kill`. Se o processo pai ainda se recusa a encerrar o zumbi, o próximo passo seria remover o processo pai. Quando um processo perder o seu pai, o `init` se torna o seu novo pai. O `init` periodicamente executa a chamada de sistema `wait` enterrando quaisquer zumbis que o possuam como pai.

Considerando a descrição acima, pede-se que o aluno modifique o programa `zombie.c` de modo que este forneça uma saída similar a apresentada para o comando `ps` na tabela abaixo (obs. são apresentadas apenas as entradas de interesse):

```
$ ps -f
```

S	PID	PPID	CMD
S	311	129	zombie
Z	312	311	<defunct>
Z	313	311	<defunct>
Z	314	311	<defunct>
Z	315	311	<defunct>
Z	316	311	<defunct>

Além disso, explique as relações de parentesco entre os processos 311, 312, 313, 314, 315 e 316.

Pergunta-se também, quem é o pai de `zombie`.

TAREFA 6: criando uma chamada de sistemas (extraclasse – recomendação 09/10/2018)

TEÓRICO: No MINIX 3 servidores tratam as chamadas de sistema. Conforme visto na unidade 1.1 (Introdução ao MINIX), existem servidores para lidar com diversos aspectos do sistema operacional (gerenciamento de processos, gerenciamento de sistema de arquivos, gerenciamento

conexões de rede etc). Para adicionar uma nova chamada de sistema são necessárias duas etapas: escrever um manipulador (*handler*) de chamada de sistema e escrever uma biblioteca de usuário. O manipulador de chamada de sistema é uma função que é chamada em resposta a um usuário solicitando uma chamada de sistema. Cada chamada de sistema tem um manipulador. Uma biblioteca do usuário empacota os parâmetros para a chamada de sistema e chama o manipulador no servidor apropriado. Um usuário sempre invoca uma chamada de sistema usando a biblioteca de usuário.

O manipulador de chamadas do sistema deve ser colocado em um servidor apropriado, que por sua vez irá processar uma solicitação do usuário, invocando o manipulador correspondente. É importante escolher o servidor correto para a inclusão da chamada de sistema. Por exemplo, se a chamada de sistema deve atualizar o sistema de arquivo ou estruturas de dados `fproc`, então o manipulador de chamada de sistema deve ser colocado no servidor FS (no MFS, para sistema de arquivos).

Para esta prática, o aluno irá adicionar uma chamada de sistema simples no MINIX 3, no servidor PM (gerenciador de processo), a qual irá imprimir a seguinte mensagem: “Eu sou uma chamada de sistema!”.

Para auxiliar o aluno, esta prática será dividida em algumas etapas – estas etapas devem constar no relatório:

- (a) O código fonte para todos os servidores está localizado em `/usr/src/servers`. Cada servidor possui um diretório separado. O gerenciador de processos (PM) está localizado em `/usr/src/servers/pm`. Cada diretório de servidor possui dois arquivos chave: `table.c` e `proto.h`. Estes arquivos contêm a definição para a tabela `call_vec`. A tabela `call_vec` é um vetor de ponteiros de função que é indexado pelo número da chamada de sistema. Em cada linha, o endereço de um manipulador de chamada de sistema é atribuído para uma entrada na tabela e o índice da entrada é o número da chamada de sistema. **Modifique os arquivos `table.c` e `proto.h` de modo a incluir o manipulador `do_mycall`. Deduza como as entradas para a criação deste manipulador devem ser inseridas nos arquivos mencionados. Use a entrada de número 70 para identificar esta chamada de sistema. Observe que `do_mycall` não recebe parâmetros de entrada. Esta chamada irá apenas imprimir a mensagem: “Eu sou uma chamada de sistema!”. Em `proto.h`, insira o protótipo de sua chamada de sistema na seção destinada ao arquivo `misc.c`.**
- (b) A implementação da chamada de sistema deve aparecer em `/usr/src/servers/pm`. Esta pode ser implementada em um arquivo “.c” a parte, porém, para simplificar o processo, iremos inserir sua implementação em `/usr/src/servers/pm/misc.c`. O

corpo da chamada deverá conter apenas:

```
printf("Eu sou uma chamada de sistema!");  
return(OK);
```

Para definir o cabeçalho da chamada de sistema, observe o cabeçalho da função `do_procstat`. Ambos são similares.

- (c) Vá ao diretório `/usr/src/servers` e faça um `make`. Isso deverá mostrar erros de compilação caso você tenha realizado alguma alteração incorretamente. Corrija seus erros caso seja necessário.
- (d) É hora de mapear a chamada de sistema e seu número (70). Para tanto, você deve ir para o diretório `/usr/src/include/minix` e editar o arquivo `callnr.h` de modo a inserir o mapeamento para a sua chamada de sistema. Chame-a de **MYCALL** no contexto desse arquivo.
- (e) Para simplificar o procedimento de criação da biblioteca de usuário, crie em `/usr/src/include` o arquivo `mylib.h`. Este arquivo irá executar efetivamente a chamada de sistema criada. O código a ser inserido nessa biblioteca é:

```
#include<lib.h>  
#define mycall      _mycall  
#include<unistd.h>  
<<similar ao cabeçalho disponível em (b)>>  
{  
    message m;  
    return (_syscall(PM_PROC_NR, MYCALL, &m));  
}
```

Não se esqueça de incluir uma entrada para `mylib.h` em `/usr/src/include/Makefile`.

- (f) Você observou no código da questão anterior a seguinte declaração: “message m”. Explique em que consiste esse tipo de dado. Explique também o papel da chamada de sistema `_syscall()` (obs. Os slides da unidade 2 possuem uma referência a esta questão. Para maiores detalhes consulte o livro Projeto de Sistemas Operacionais o qual consta na referência do curso).
- (g) Para finalizar o processo de criação de sua chamada de sistema, vá ao diretório `/usr/src/tools` compile o kernel, bibliotecas e includes e instale no HD.
- (h) Crie um programa de teste para sua nova chamada de sistema. **Dica:** você teria dificuldade em demonstrar a chamada de sistema `printf()`? O raciocínio é análogo.

TAREFA 7: criando uma chamada de kernel (extraclasse – recomendação: 09/10/2018)

Esta tarefa foi projetada para guiar o aluno no procedimento para a confecção de uma chamada de kernel. Para auxiliar o aluno, esta prática será dividida em algumas etapas – estas etapas devem constar no relatório:

- (a) Adicione o protótipo de uma função de kernel chamada `do_mykernelcall()` no arquivo: `/usr/src/kernel/system.h`

```
...  
int do_mykernelcall(struct proc *caller, message *m_ptr);  
  
#endif /* SYSTEM_H */
```

- (b) Escreva a implementação de `do_mykernelcall()` em seu próprio arquivo fonte:

```
/usr/src/kernel/system/do_mykernelcall.c  
#include "kernel/system.h"  
#include <minix/endpoint.h>  
  
/*===== *  
* do_mykernelcall *  
*=====*/  
int do_mykernelcall(struct proc *caller_ptr, message *m_ptr)  
{  
    printf ("Esta é minha primeira chamada de kernel!");  
    return(OK);  
}
```

- (c) Adicione `do_mykernelcall` ao `Makefile` para compilação:

```
/usr/src/kernel/system/Makefile.inc  
# Makefile for system library implementation  
.include <bsd.own.mk>  
  
.PATH: ${.CURDIR}/system  
SRCS+= \  
    do_fork.c \  
    do_exec.c \  
...
```

```
do_statectl.c \
do_mykernelcall.c
```

- (d) Mapeie `SYS_MYKERNELCALL` a `do_mykernelcall()` na tabela de chamadas de sistema: `/usr/src/kernel/system.c`

```
...
map(SYS_STATECTL, do_statectl); /* let a process control its
                                state */
map(SYS_MYKERNELCALL, do_mykernelcall); /* your kernel call
                                         */
...
```

- (e) Adicione um protótipo para a função `sys_mykernelcall` no arquivo: `/usr/src/include/minix/syslib.h`

```
...
int sys_mykernelcall(unsigned flags, endpoint_t proc_ep);
...
```

- (f) Adicione o número de chamada para `sys_mykernelcall` ao vetor de chamada e incremente sua dimensão: `/usr/src/include/minix/com.h`

```
...
#define SYS_STATECTL (KERNEL_CALL + 55) /* sys_statectl() */
#define SYS_MYKERNELCALL (KERNEL_CALL + 56) /*
sys_mykernelcall() */

/* Total */
#define NR_SYS_CALLS 57 /* number of kernel calls */
...
```

- (g) Adicione o serviço `MYKERNELCALL` à tabela de serviços do sistema: `/usr/src/commands/service/service.c`;

construa a tabela de serviços do sistema e instale-a com `make`; `make install` a partir de `/usr/src/commands/service/`

```
...
struct
{
    char *label;
    int call_nr;
} system_tab[]=
```

```

{
...
    { "MYKERNELCALL",          SYS_MYKERNELCALL },
    { NULL,                   0 }
};
...

```

- (h) Escreva sua implementação da função `sys_mykernelcall` em um novo arquivo:

```

/usr/src/lib/libsys/sys_mykernelcall.c
#include "syslib.h"

int sys_mykernelcall(unsigned flags, endpoint_t proc_ep)
{
    message m;

    return(_kernel_call(SYS_MYKERNELCALL, &m));
}

```

- (i) Adicione `sys_mykernelcall.c` ao Makefile, e instale-a em `/usr/lib/${MACHINE_ARCH}` com `make install`.

```

# Makefile for libsys

LIB=          sys
SRCS=  \
...
    sys_mykernelcall.c \
...

```

- (j) Teste a nova chamada de kernel usando a chamada de sistema do exercício anterior.

Adicione a seguinte linha à implementação da chamada de sistema `do_mycall()`

```

_kernel_call (SYS_MYKERNELCALL, &m);

```

- (k) Reconstrua o kernel e instale-o novamente com `make`; `make install` a partir de `/usr/src/tools`.

TAREFA 8 (Entrada/Saída): criando um controlador de dispositivo simples (*device driver*) (extraclasse – recomendação: 11/10/2018)

Esta prática tem o intuito de apresentar as bases para a programação de controladores de

dispositivos no Minix usando linguagem C. Um *device driver* é um programa de computador, o qual interage com componentes de hardware reais. Por exemplo, o computador possui *device drivers* para dispositivos como o monitor, disco rígido etc.

Requisitos

O desenvolvimento de um *device driver* requer habilidades de programação e conhecimento do dispositivo alvo. Nesta prática, assume-se que o aluno possui conhecimentos básicos de linguagem C e que já tenha trabalhado algum tempo com o sistema Minix. Para esta prática, será estudado um *device driver* bastante simples; o 'Olá, Mundo!'. Este *device driver* possui apenas uma tarefa: imprimir uma mensagem de olá em seu início e então terminar.

Observe que este *device driver* já existe e pode ser encontrado em `/usr/src/drivers/hello`. O resto deste documento irá assumir que o *device driver* não está lá e você irá criá-lo por conta própria.

(a) O primeiro passo é criar um diretório na árvore de diretórios do Minix para colocar o código fonte do seu *driver*. Como *root* execute:

```
# cd /usr/src/drivers
# mkdir ola
# cd ola
```

Para compilar o *device driver* é necessário criar um Makefile. Você pode usar o seguinte código em seu **Makefile**:

```
# Makefile for the hello driver.
PROG=    ola
SRCS=    ola.c
```

```
DPADD+=  ${LIBDRIVER} ${LIBSYS}
LDADD+=  -ldriver -lsys
```

```
MAN=
```

```
BINDIR?= /usr/sbin
```

```
.include <bsd.prog.mk>
```

Posteriormente, crie o arquivo `ola.c` para o código do *driver*:

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <minix/syslib.h>

int main(int argc, char **argv)
{
    sef_startup();

    printf("Olá, mundo!\n");
    return EXIT_SUCCESS;
}
```

Ignore a chamada `sef_startup()` por agora. Esta será explicada posteriormente. **Teste a compilação deste código.**

```
# make clean
# make
# make install
```

Se você não recebeu quaisquer erros, é o momento de tentar executar o *driver*. Antes de fazer isso, devemos modificar o arquivo `/etc/system.conf`. Ele contém uma lista de todos os serviços de sistema (servidores e *drivers*) e suas permissões. Cada *device driver* precisa normalmente acessar um dispositivo de hardware real e usa algumas poucas funções oferecidas pelo Minix. Para testar o *driver* permissões suficientes, teste-o com a seguinte configuração no `/etc/system.conf`:

```
service ola
{
    system
        IRQCTL          # 19
        DEVIO           # 21
    ;
    ipc
        SYSTEM pm rs log tty ds vm vfs
        pci inet amddev
    ;
    uid 0;
};
```

O procedimento de iniciar (*start*), parar (*stop*) e reiniciar (*restart*) *device drivers* deve ser realizado

usando o comando *service* (8). Para iniciar o programa *ola*, digite o seguinte comando:

```
# service up /usr/sbin/ola
```

Olá, mundo!

No terminal 1, aparece a mensagem **Olá, mundo!** Caso deseje verificar a mensagem usando o arquivo de mensagens do sistema (provenientes do kernel) use: **tail -f /var/log/messages**

Para parar o *driver* use:

```
# service down ola
```

OBS. Em versões mais recentes do Minix, o *Reincarnation Server* (RS) envia mensagens para a operação de início e parada do *driver*. Infelizmente, para a versão usada durante esse curso, isso não acontece.

Como o Minix opera como um *microkernel*, os *device drivers* são programas separados os quais enviam e recebem mensagens para se comunicar com outros componentes de sistema. *Device drivers*, como qualquer outro programa, podem conter *bugs* e podem sofrer falhas a qualquer momento. O RS tentará reiniciar os *device drivers* ao perceber que são abruptamente terminados pelo kernel devido a uma falha (*crash*), ou em caso de uma chamada `exit(2)` inesperada. O RS pode ser visto na lista de processos como *rs*, ao usar o comando *ps*(1). O RS envia mensagens *keep-a-live* para cada *device driver* em execução no sistema, periodicamente, para assegurar que eles ainda estejam responsivos, e não parados em um laço infinito.

O *driver ola* responde às mensagens *keep-a-live*, de modo que o RS detecta corretamente se este ainda está em execução. Felizmente, como veremos no próximo exercício, existe uma biblioteca, a *libdriver* (ou *libchardriver* em versões mais recentes) no Minix. Esta biblioteca lida com várias tarefas comuns a todos os *character devices*. Por exemplo, desempenha a comunicação com o RS e os componentes do SO de forma transparente.

Por outro lado, uma funcionalidade que é diretamente exposta ao desenvolvedor de *device driver* é o protocolo de inicialização. Quando um novo *device driver* (ou qualquer outro serviço de sistema) é iniciado, o RS irá enviar ao *driver* uma mensagem de inicialização com informação sobre como ser inicializado adequadamente. Espera-se que o *driver* responda com um OK (inicialização completada de forma bem sucedida) ou com um erro (inicialização falhou). No primeiro caso, o RS assumirá que o novo *device driver* foi inicializado corretamente. No último caso, o RS irá imediatamente terminar o *driver* sem tentar reiniciá-lo. O usuário será informado com uma

mensagem no console (console 1). Felizmente, o protocolo de inicialização é completamente oculto no **System Event Framework (SEF)**, o qual expõe chamadas de biblioteca as quais deixam que os desenvolvedores tratem a inicialização de forma simples e efetiva.

No início do código *main()*, o desenvolvedor precisa registrar uma ou mais chamadas de tratamento (*callbacks*) e então deixam o SEF realizar o restante pela chamada de *sef_startup()*. Esta é a razão pela qual inserimos uma chamada *sef_startup()* no código supracitado. Cada chamada de *callback* nada mais é do que uma função que recebe dados de inicialização como entrada e retorna um código de *status* para determinar o resultado do processo de inicialização. Por agora, os desenvolvedores podem registrar *callbacks* para três tipos de inicializações: *fresh* (quando um serviço é iniciado pela primeira vez), *live update* (quando o serviço é dinamicamente atualizado para uma nova versão), *restart* (quando um serviço é reiniciado após uma falha (*crash*) ou um reinício controlado).

(b) Criação de um *device* /dev/ola

Neste exercício, você irá estender o *driver ola* e re-implementá-lo usando a *libdriver* (*libchardriver* na versão 3.2.0). Ao invés de apenas imprimir uma mensagem 'Olá' no início, agora iremos usar um arquivo de dispositivo /dev/hello para ler a mensagem 'Olá, mundo!'. Cada *driver* de caractere e bloco é associado com um número de dispositivo maior ordem (*major*). Assim, é necessário escolher um número de dispositivo *major* livre para o dispositivo. O número *major* usado será o 17 para ID do dispositivo. Para começar, criamos o arquivo de dispositivo:

```
# mknod /dev/ola c 17 0
```

Explique exatamente o que este comando realiza em seu relatório.

Agora, reutilizaremos o Makefile da letra (a). Crie um arquivo de cabeçalho *ola.h*:

```
#ifndef __OLA_H
#define __OLA_H

/** A mensagem 'Ola, mundo!'. */
#define MENSAGEM_OLA "Ola, mundo!\n"

#endif /* __OLA_H */
```

Este arquivo possui uma macro de pré-processamento para a mensagem. Agora coloque o arquivo no mesmo diretório de *ola.c* e, acrescente-o dentre os seus arquivos de cabeçalho. Nosso código C, *ola.c* será modificado como segue (copie e cole /usr/src/drivers/hello/hello.c e modifique):


```

#include <minix/drivers.h>
#include <minix/driver.h>
#include <stdio.h>
#include <stdlib.h>
#include <minix/ds.h>
#include "ola.h"

/*
 * Prototipos de funcao para o driver ola.
 */
FORWARD _PROTOTYPE( char * ola_name,    (void) );
FORWARD _PROTOTYPE( int  ola_open,    (struct driver *d, message
*m) );
FORWARD _PROTOTYPE( int  ola_close,    (struct driver *d, message
*m) );
FORWARD _PROTOTYPE( struct device * ola_prepare, (int device) );
FORWARD _PROTOTYPE( int ola_transfer, (      int procnr,
                                           int opcode,
                                           u64_t position,
                                           iovec_t *iov,
                                           unsigned nr_req) );
FORWARD _PROTOTYPE( void  ola_geometry, (struct partition
*entry) );

/* SEF funcoes e variaveis. */
FORWARD _PROTOTYPE( void sef_local_startup, (void) );
FORWARD _PROTOTYPE( int  sef_cb_init, (int type, sef_init_info_t
*info) );
FORWARD _PROTOTYPE( int  sef_cb_lu_state_save, (int) );
FORWARD _PROTOTYPE( int  lu_state_restore, (void) );

/* Pontos de entrada para o driver ola. */
PRIVATE struct driver ola_tab =
{
    ola_name,
    ola_open,

```

```

    ola_close,
    nop_ioctl,
    ola_prepare,
    ola_transfer,
    nop_cleanup,
    ola_geometry,
    nop_alarm,
    nop_cancel,
    nop_select,
    nop_ioctl,
    do_nop,
};

/** Representa o dispositivo /dev/ola */
PRIVATE struct device ola_device;

/** Variavel de estado para contar o numero de vezes que o
dispositivo foi aberto. */
PRIVATE int open_counter;

PRIVATE char * ola_name(void)
{
    printf("ola_name()\n");
    return "ola";
}

PRIVATE int ola_open(d, m)
    struct driver *d;
    message *m;
{
    printf("ola_open(). Chamou %d vez(es).\n", ++open_counter);
    return OK;
}

PRIVATE int ola_close(d, m)
    struct driver *d;

```

```

    message *m;
{
    printf("ola_close()\n");
    return OK;
}

PRIVATE struct device * ola_prepare(dev)
    int dev;
{
    ola_device.dv_base.lo = 0;
    ola_device.dv_base.hi = 0;
    ola_device.dv_size.lo = strlen(MENSAGEM_OLA);
    ola_device.dv_size.hi = 0;
    return &ola_device;
}

PRIVATE int ola_transfer(proc_nr, opcode, position, iov, nr_req)
    int proc_nr;
    int opcode;
    u64_t position;
    iovec_t *iov;
    unsigned nr_req;
{
    int bytes, ret;

    printf("ola_transfer()\n");

    bytes = strlen(MENSAGEM_OLA) - position.lo < iov->iov_size ?
        strlen(MENSAGEM_OLA) - position.lo : iov->iov_size;

    if (bytes <= 0)
    {
        return OK;
    }
    switch (opcode)
    {

```

```

        case DEV_GATHER_S:
            ret = sys_safecopyto(proc_nr, iov->iov_addr, 0,
                                (vir_bytes) (MENSAGEM_OLA +
position.lo), bytes, D);
            iov->iov_size -= bytes;
            break;

        default:
            return EINVAL;
    }
    return ret;
}

```

```

PRIVATE void ola_geometry(entry)
    struct partition *entry;
{
    printf("ola_geometry()\n");
    entry->cylinders = 0;
    entry->heads      = 0;
    entry->sectors     = 0;
}

```

```

PRIVATE int sef_cb_lu_state_save(int state) {
/* Salva o estado. */
    ds_publish_u32("open_counter", open_counter, DSF_OVERWRITE);

    return OK;
}

```

```

PRIVATE int lu_state_restore() {
/* Restaura o estado. */
    u32_t value;

    ds_retrieve_u32("open_counter", &value);
    ds_delete_u32("open_counter");
    open_counter = (int) value;
}

```

```

        return OK;
    }

PRIVATE void sef_local_startup()
{
    /*
     * Registra chamadas de callback init. Usa a mesma funcao para
    todos os tipos de evento
     */
    sef_setcb_init_fresh(sef_cb_init);
    sef_setcb_init_lu(sef_cb_init);
    sef_setcb_init_restart(sef_cb_init);

    /*
     * Registra chamadas de live update.
     */
    /* - Agree to update immediately when LU is requested in a
    valid state. */
    sef_setcb_lu_prepare(sef_cb_lu_prepare_always_ready);
    /* - Support live update starting from any standard state. */
    sef_setcb_lu_state_isvalid(sef_cb_lu_state_isvalid_standard);
    /* - Register a custom routine to save the state. */
    sef_setcb_lu_state_save(sef_cb_lu_state_save);

    /* Let SEF perform startup. */
    sef_startup();
}

PRIVATE int sef_cb_init(int type, sef_init_info_t *info)
{
    /* Inicia o driver alo. */
    int do_announce_driver = TRUE;

    open_counter = 0;
    switch(type) {

```

```

        case SEF_INIT_FRESH:
            printf("%s", MENSAGEM_OLA);
            break;

        case SEF_INIT_LU:
            /* Restore the state. */
            lu_state_restore();
            do_announce_driver = FALSE;

            printf("%sEi, Eu sou uma nova versao!\n",
MENSAGEM_OLA);
            break;

        case SEF_INIT_RESTART:
            printf("%sEi, acabo de ser reiniciado!\n",
MENSAGEM_OLA);
            break;
    }

    /* Anuncia que esta online quando necessario. */
    if (do_announce_driver) {
        driver_announce();
    }

    /* Inicializacao bem sucedida. */
    return OK;
}

PUBLIC int main(int argc, char **argv)
{
    /*
     * Realiza inicializacao.
     */
    sef_local_startup();

    /*

```

```

    * Executa o laço principal.
    */
    driver_task(&ola_tab, DRIVER_STD);
    return OK;
}

```

Agora, vamos entender o que o código acima realiza. Primeiro, ele possui várias linhas de `#include` para os protótipos e funções necessários, usados no programa. Posteriormente, declara-se uma *struct driver*. Esta estrutura é preenchida com funções de *callback* as quais serão invocadas pelo *libdriver* (ou *libchardriver* na versão $\geq 3.2.0$) em tempo de execução. Ela possui funções de *callback* para realizar operações de *open*, *read*, *write* e *close* no *device driver*. A maior parte da ação ocorre na função *ola_transfer*. Ela é usada pela função *sys_safecopyto* para copiar a *string* `MENSAGEM_OLA` do programa *device driver* para o programa de usuário lendo o `/dev/ola`. O sistema operacional irá então, por parte do *device driver*, tomar conta de copiar os bytes entre os dois programas adequadamente. Um vetor de entrada/saída *iov* é usado pela função *sys_safecopyto* para descrever o endereço de memória para armazenar bytes e o número de bytes requisitados pela aplicação do usuário.

Na função *main()*, existem apenas duas chamadas simples. *sef_local_startup()* é chamada no início para registrar as chamadas de *callback* SEF e então completar a inicialização. Em *sef_local_startup()* a mesma função *sef_cb_init_fresh()* é registrada como uma função de *callback* para todos os tipos de inicialização suportados. Isto significa que o código de inicialização executado quando o *driver* inicia será sempre o mesmo independente do *device driver* iniciado, após um *live update*, ou após um reinício.

No *sef_local_startup()*, também registramos *callbacks* para eventos *live update*. Um evento *live update* é disparado pelo RS quando uma nova versão de um serviço de sistema está disponível. Neste ponto, o RS envia uma mensagem para a versão mais antiga do serviço pedindo para se preparar para um estado particular como requerido pela atualização (*update*). O serviço irá decidir se irá aceitar o estado de *update* solicitado ou rejeitar o *update*. No primeiro caso, o serviço se compromete a se preparar para a atualização e responde ao RS quando o estado alvo foi alcançado. Somente neste ponto, o RS instala a atualização e a nova versão pode ser iniciada a partir do estado onde a versão mais antiga parou. Felizmente, o protocolo de atualização (*live update*) está escondido do SEF e o desenvolvedor de *driver* não precisa se preocupar com todos os detalhes. Para suportar o *live update* para um serviço do sistema, o desenvolvedor deve apenas registrar *callbacks*, para deixar o SEF saber o que fazer quando uma requisição de *live update* chega. Os tipos de chamada de *callback* mais importantes para um *live update* são *state_isvalid* e *prepare*. Uma chamada de *callback* *state_isvalid* deve ser registrada para informar o SEF quais estados de *update*

devem ser aceitos. Uma chamada de *callback prepare*, por outro lado, é chamada toda vez que um serviço de sistema é recebe uma chance de se preparar para a atualização (após uma solicitação de *update* ser aceita). A chamada de *callback* deve pegar o estado do alvo como entrada e retornar um OK somente se o serviço estiver pronto para a atualização solicitada.

O *driver ola* é completamente *stateless* e pode suportar *live update* começando a partir de qualquer estado e deixa o RS realizar o *update* na primeira chance possível. Para completar, registramos duas chamadas de *callback* pré-definidas para os tipos *state_isvalid* e *prepare*. Estas são as funções de biblioteca implementadas dentro do SEF *sef_cb_lu_state_isvalid_standard* e *sef_cb_lu_prepare_always_ready*, respectivamente. A chamada de *callback sef_cb_lu_state_isvalid_standard* aceita qualquer padrão *live update* e a *callback sef_cb_lu_prepare_always_ready* informa que o serviço está pronto e atualizado. Quando o *device driver* não é *stateless*, o desenvolvedor precisa fornecer as implementações apropriadas para estes dois tipos de *callback*, para suportar os estados padrão e possivelmente alguns outros estados particulares definidos pelo *driver*. Os estados padrão definidos são SEF_LU_STATE_WORK_FREE (o serviço não está fazendo nenhum trabalho), SEF_LU_STATE_REQUEST_FREE (o serviço não possui requisição pendente), SEF_LU_STATE_PROTOCOL_FREE (o serviço não está envolvido em um protocolo).

Finalmente, a função *main()* executa *driver_task()*(ou *chardriver_task()*) para deixar o *libdriver* (ou *libchardriver*) começar a processar requisições de usuário e invocar as funções de *callback* do *driver ola*.

Para finalizar, compile o programa novamente usando os mesmos comandos apresentados no item (a), e inicie o driver com so comando *service*. Acrescente '*-dev /dev/ola*') para indicar que o *driver* iniciado no momento é responsável pelo dispositivo identificado por */dev/ola* – no caso, *major 17*, o qual foi escolhido anteriormente.

```
# service up /usr/sbin/ola -dev /dev/ola
Ola, mundo!
```

Perceba que não há mais reinício do RS (múltiplas ocorrências da mensagem). Agora tente verificar se é possível ler a mensagem a partir do arquivo de dispositivo. É possível fazer isso, simplesmente usando o comando *cat(1)*.

```
# cat /dev/ola
ola_open()
ola_transfer()
ola_transfer()
ola_close()
```


Ola, mundo!

#

Se você receber a mensagem acima, o *driver ola* funcionou corretamente.

Agora vamos realizar o reinício do *driver* com o comando *service(8)* para simular uma falha:

```
# serscavice refresh ola
```

```
Ei, acabei de ser reiniciado!
```

```
# cat /dev/ola
```

```
ola_open()
```

```
ola_transfer()
```

```
ola_transfer()
```

```
ola_close()
```

```
Ola, mundo!
```

#

Finalmente, teste um *live update*. Faça a seguinte mudança no arquivo de cabeçalho *ola.h*:

```
#ifndef __OLA_H
```

```
#define __OLA_H
```

```
/** A mensagem 'Ola, mundo!'. */
```

```
#define MENSAGEM_OLA "Ola, novo mundo!\n"
```

```
#endif /* __OLA_H */
```

Agora recompile o *driver* e atualize-o com o comando *service(8)*:

```
# make clean install
```

```
# service update /usr/sbin/ola -state 1
```

```
Ola, novo mundo!
```

```
Ei, eu sou uma nova versao!
```

Com isso, se solicita uma atualização de estado onde nenhum trabalho esteja em progresso (i.e.

SEF_LU_STATE_WORK_FREE=1 – esse é o padrão)

Executando o comando *cat*:

```
# cat /dev/ola
```

```
ola_open()
```

```
ola_transfer()
```

```
ola_transfer()  
ola_close()  
Ola, novo mundo!  
#
```

Como esperado, o *driver* é atualizado ao vivo, e a nova versão é imediatamente iniciada.